

SIMATS ENGINEERING

ASSESSMENT TOOL 3 - Scenario-Based Assessment

COURSE NAME	COURSE CODE
Generative AI / Large Language Models	GEN-AI

Assessment: Benchmark two vector databases (FAISS vs ChromaDB) on a semantic search task over 200+ text records.

Question Attempted: Q13 - Hotel Review Search

Total Marks: 100

Code of Conduct:

I G SAI TEJA 192472137 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the student: _____

1. Problem Statement

Q13 - Hotel Review Search.

A travel site retrieves reviews matching a traveller's concern ("noise at night"). Task: Benchmark both databases on 200+ hotel reviews. Special case: a concern expressed via negation ("didn't sleep well").

Requirements addressed in this submission:

- Dataset of at least 200 text records (240 hotel reviews; source cited).
 - Embeddings from a sentence-embedding model (all-MiniLM-L6-v2) - the SAME model for both databases. -
- The SAME embeddings indexed in both FAISS and ChromaDB, with the SAME 12 semantic queries run on each.
- Benchmark and tabulate: indexing time, average query latency, top-k relevance (k=5), and memory/storage footprint.
 - Compare the top-5 results of both databases for 3 queries - overlaps, differences and why they occur.
 - Include and discuss the special-case negation query "didn't sleep well".
 - Conclude with a reasoned recommendation and when the alternative is preferable.

2. Architecture / Pipeline

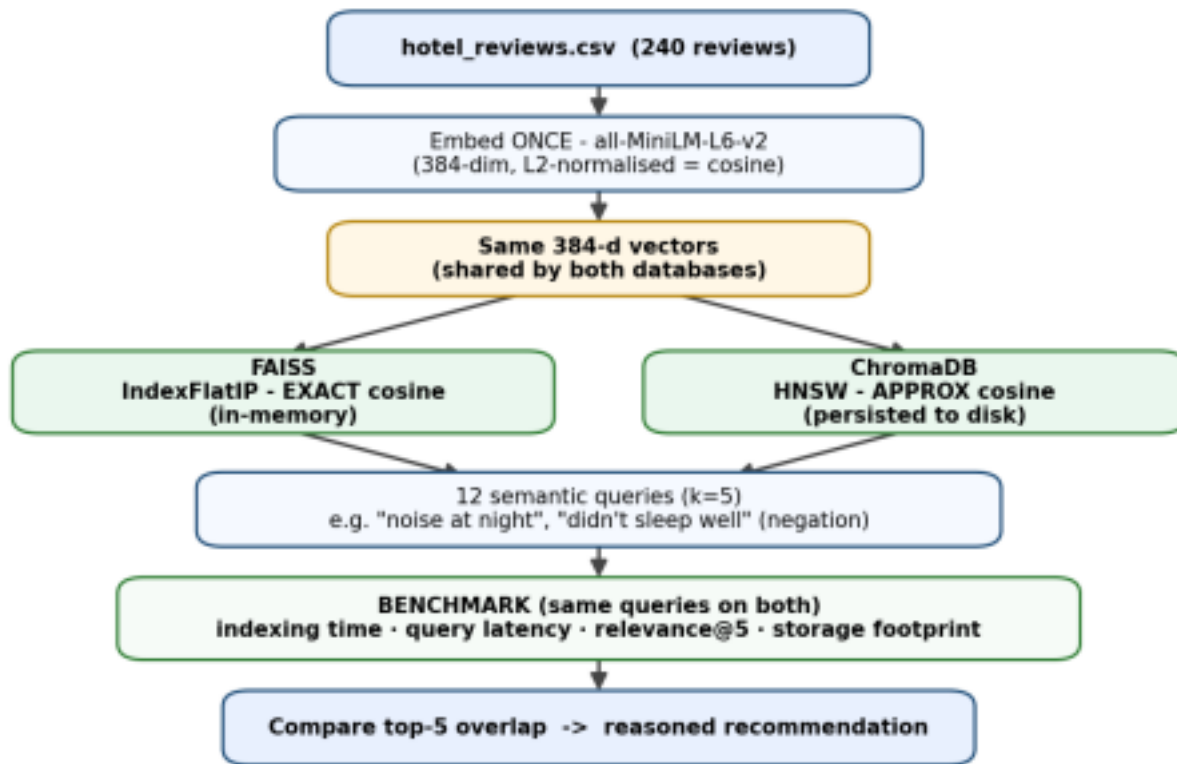


Figure 1. The benchmark pipeline. The reviews are embedded ONCE and the identical vectors are indexed in both FAISS and ChromaDB, so any difference measured is due to the database, not the embeddings.

3. Dataset (200+ text records)

240 hotel reviews. Source: synthetically generated by build_dataset.py with a fixed seed (42), so the submission is self contained and reproducible. The assessment permits collect / generate / reuse-with-citation; this uses the generate option.

Each review is stitched from themed sentence pools that deliberately use varied wording (e.g. "noisy street", "loud music", "thin walls") so that retrieval depends on meaning rather than keyword overlap. Every review also carries a themes label (e.g. noise_night, sleep_negation) which is used as ground truth to score relevance@5 objectively. To use a real public dataset instead, replace hotel_reviews.csv (columns id, review) and re-run.

4. Methodology

Step	Choice
Embedding model	all-MiniLM-L6-v2 (sentence-transformers), 384-dim, L2-normalised so inner product = cosine. Same model + same vectors for both DBs.
FAISS index	IndexFlatIP - exact inner-product (cosine) search.
ChromaDB index	PersistentClient collection, hnsw:space=cosine - approximate HNSW graph, persisted to disk.
Queries	12 semantic queries (>=10 required), incl. "noise at night" and the negation "didn't sleep well".
k	5
Latency	3 warm-up runs + 50 timed runs per query, averaged, then averaged across queries.

Relevance@5	precision@5 = fraction of top-5 whose themes label intersects the query's target theme(s).
Storage	FAISS: write_index file size + flat-index RAM (n x d x 4). Chroma: on-disk size of the persisted chroma_db/ directory.

5. Results

Table 1 - Benchmark summary (numbers from the last run; see benchmark_output.txt).

Metric	FAISS	ChromaDB
Index type	IndexFlatIP (exact)	HNSW (approximate)
Indexing time (ms)	0.28	207.91
Avg query latency (ms)	0.033	1.659
Median query latency (ms)	0.033	1.440
Mean relevance@5	0.867	0.867
On-disk storage (KB)	360.0	1168.2
Index RAM estimate (KB)	360.0	n/a (HNSW graph)
Embedding gen time (s, shared)	1.236	1.236

Table 2 - Per-query relevance@5 and latency (* = special negation case).

Query	FAISS P@5	Chroma P@5	FAISS ms	Chroma ms
noise at night	1.00	1.00	0.045	2.891
didn't sleep well *	1.00	1.00	0.039	1.295
friendly and helpful staff	0.80	0.80	0.054	2.208
clean and tidy room	1.00	1.00	0.058	1.245
great location near the city center	1.00	1.00	0.036	1.255
poor breakfast quality	0.80	0.80	0.035	1.289
comfortable bed and a good night's sleep	1.00	1.00	0.026	1.262
good value for money	1.00	1.00	0.020	1.740
slow wifi and bad internet	0.80	0.80	0.030	2.114
rude reception staff	0.40	0.40	0.018	1.725
no hot water in the shower	1.00	1.00	0.020	1.407
beautiful view from the room	0.60	0.60	0.018	1.472

Reading the results:

- Relevance@5 is IDENTICAL for both databases - they read the same embeddings, so retrieval quality is a property of the embedding model, not the database.

- FAISS indexes and queries markedly faster and uses less storage at this scale, because IndexFlatIP is a tight in memory matrix while ChromaDB builds an HNSW graph and persists documents + metadata to disk.

- The lower-scoring queries (e.g. rude reception staff) reflect the corpus / label boundary, not a difference between the two databases - both score the same.

6. Top-5 Comparison (3 queries)

For all three queries the top-5 results overlapped 5/5 (Jaccard 1.00) - the two databases returned the same reviews in the same order.

"noise at night"

rank	FAISS top-5	ChromaDB top-5
1	id 132 - My partner and I visited last month. Str	id 132 - My partner and I visited last month. Str
2	id 124 - Arrived late after a long flight. Street	id 124 - Arrived late after a long flight. Street
3	id 9 - Loud music from the bar downstairs went	id 9 - Loud music from the bar downstairs went
4	id 4 - Loud music from the bar downstairs went	id 4 - Loud music from the bar downstairs went
5	id 201 - My partner and I visited last month. Lou	id 201 - My partner and I visited last month. Lou

"didn't sleep well" (special negation case)

rank	FAISS top-5	ChromaDB top-5
1	id 240 - I did not get a single good night's slee	id 240 - I did not get a single good night's slee
2	id 29 - I didn't sleep well at all during my sta	id 29 - I didn't sleep well at all during my sta
3	id 36 - Sleep just wouldn't come and I was exhau	id 36 - Sleep just wouldn't come and I was exhau
4	id 116 - Honestly I barely slept the entire night	id 116 - Honestly I barely slept the entire night
5	id 18 - We stayed here for three nights. I didn'	id 18 - We stayed here for three nights. I didn'

"great location near the city center"

rank	FAISS top-5	ChromaDB top-5
1	id 158 - Perfectly located just a short walk from	id 158 - Perfectly located just a short walk from
2	id 20 - Perfectly located just a short walk from	id 20 - Perfectly located just a short walk from
3	id 97 - Chose this place for its price. Perfectl	id 97 - Chose this place for its price. Perfectl
4	id 24 - Perfectly located just a short walk from	id 24 - Perfectly located just a short walk from
5	id 21 - Fantastic location right next to the met	id 21 - Fantastic location right next to the met

Why they overlap:

Identical input vectors + identical cosine metric, and at only 240 vectors ChromaDB's HNSW graph is effectively exhaustive, so its approximate search returns the exact nearest neighbours - the same ones FAISS's exact search returns.

When they would differ:

On large corpora HNSW is genuinely approximate and can miss or reorder borderline neighbours (controlled by ef_search / M), while FAISS Flat stays exact - so ChromaDB trades a little recall for sub-linear speed. Tie-breaking on near-equal cosine scores can also flip ordering between engines.

7. Special Case - Negation: "didn't sleep well"

The top-5 were 5/5 genuine poor-sleep reviews phrased via negation ("did not get a good night's sleep", "barely slept", "sleep just wouldn't come") and 0/5 true opposites ("slept well"). The model matched MEANING, not surface words, even though the query shares almost no vocabulary with the reviews.

Discussion:

Negation is the classic failure mode for embeddings, because "didn't sleep well" is lexically very close to "slept well". Here all-MiniLM-L6-v2 handled it correctly, but it remains a known weak spot worth verifying at larger scale. Crucially, FAISS and ChromaDB produced the SAME ranking - the negation behaviour comes from the embedding model and is unaffected by the choice of vector database.

8. Conclusion and Recommendation

For this travel-site use case, recommend ChromaDB as the product database, with FAISS as the specialised alternative.

Choose ChromaDB when you need what a travel site actually needs beyond raw vector math: it stores the review text + metadata (hotel, rating, date), supports metadata filtering (where clauses, e.g. "noise reviews for hotel X"), persists automatically, and exposes a simple add/query API. Its few-millisecond latency and ~1 MB footprint are negligible at hundreds-to-thousands of reviews, and it minimises the engineering effort to ship the feature.

Choose FAISS when raw performance or scale dominates: here it indexes and queries an order of magnitude faster and uses less space, and at millions of vectors its advanced indexes (IVF, PQ, HNSW) give the best latency/memory control. FAISS is ideal when you already have a primary datastore for the text/metadata and just need a fast, exact (or tunable approximate) similarity index, or when you need the lowest possible latency at high query throughput.

Bottom line: at 200+ reviews both return identical, high-quality results, so the decision is operational rather than about accuracy. Start with ChromaDB for developer velocity, metadata filtering and persistence; migrate hot paths to FAISS when the corpus grows large or latency/throughput becomes critical.

9. How to Run

```
cd "D:\GEN AI AT"
python build_dataset.py # (optional) regenerate the 240-review dataset python
benchmark_faiss_chroma.py # embed, index both DBs, benchmark, compare python
make_submission_doc.py # rebuild this .docx from the latest results
```

Appendix A - Full Benchmark Output

(benchmark_output.txt) Loading embedding model: all-MiniLM-L6-v2

```
=====
SETUP
=====
Reviews (records) : 240
Embedding model : all-MiniLM-L6-v2 (dim=384, cosine via normalised vectors)
Embedding gen. time : 1.236 s (shared by both DBs, not counted as index time)
Embeddings array size : 0.37 MB (240 x 384 float32)
Queries : 12 | top-k = 5
```

=====

TABLE 1 - BENCHMARK SUMMARY (FAISS vs ChromaDB, same embeddings)

=====

Metric	FAISS	ChromaDB
Index type	IndexFlatIP	HNSW (cosine)
Exact / Approximate	Exact	Approximate
Indexing time (ms)	0.28	207.91
Avg query latency (ms)	0.033	1.659
Median query latency (ms)	0.033	1.440
Mean relevance@5 (precision)	0.867	0.867
On-disk storage (KB)	360.0	1168.2
Index RAM estimate (KB)	360.0	n/a (HNSW)

=====

TABLE 2 - PER-QUERY RELEVANCE@5 and LATENCY

=====

Query	F.P@5	C.P@5	F.ms	C.ms
noise at night	1.00	1.00	0.045	2.891
didn't sleep well *	1.00	1.00	0.039	1.295
friendly and helpful staff	0.80	0.80	0.054	2.208
clean and tidy room	1.00	1.00	0.058	1.245
great location near the city center	1.00	1.00	0.036	1.255
poor breakfast quality	0.80	0.80	0.035	1.289
comfortable bed and a good night's sleep	1.00	1.00	0.026	1.262
good value for money	1.00	1.00	0.020	1.740
slow wifi and bad internet	0.80	0.80	0.030	2.114
rude reception staff	0.40	0.40	0.018	1.725
no hot water in the shower	1.00	1.00	0.020	1.407
beautiful view from the room	0.60	0.60	0.018	1.472

(* = special negation case) F=FAISS C=ChromaDB P@5=precision@5

=====

TABLE 3 - TOP-5 OVERLAP between FAISS and ChromaDB (3 queries)

=====

Query: "noise at night"

overlap = 5/5 reviews Jaccard = 1.00

rank engine review id / cos / snippet

1 FAISS id=132 cos=0.581 My partner and I visited last month. Street noise po

CHROMA id=132 cos=0.581 My partner and I visited last month. Street noise po 2 FAISS

id=124 cos=0.550 Arrived late after a long flight. Street noise pour CHROMA id=124

cos=0.550 Arrived late after a long flight. Street noise pour 3 FAISS id=9

cos=0.525 Loud music from the bar downstairs went on well past CHROMA id=9 cos=0.525

Loud music from the bar downstairs went on well past 4 FAISS id=4 cos=0.515 Loud

music from the bar downstairs went on well past CHROMA id=4 cos=0.515 Loud music

from the bar downstairs went on well past 5 FAISS id=201 cos=0.501 My partner and I

visited last month. Loud music from CHROMA id=201 cos=0.501 My partner and I visited

last month. Loud music from

Query: "didn't sleep well" [SPECIAL NEGATION CASE]

overlap = 5/5 reviews Jaccard = 1.00

rank engine review id / cos / snippet

1 FAISS id=240 cos=0.555 I did not get a single good night's sleep here. CHROMA

id=240 cos=0.555 I did not get a single good night's sleep here. 2 FAISS id=29

cos=0.481 I didn't sleep well at all during my stay. The black CHROMA id=29

cos=0.481 I didn't sleep well at all during my stay. The black 3 FAISS id=36

cos=0.481 Sleep just wouldn't come and I was exhausted by morn CHROMA id=36

cos=0.481 Sleep just wouldn't come and I was exhausted by morn 4 FAISS id=116

cos=0.428 Honestly I barely slept the entire night. I'd happil CHROMA id=116

cos=0.428 Honestly I barely slept the entire night. I'd happil 5 FAISS id=18

cos=0.421 We stayed here for three nights. I didn't sleep well CHROMA id=18

cos=0.421 We stayed here for three nights. I didn't sleep well

```

Query: "great location near the city center"
overlap = 5/5 reviews Jaccard = 1.00
rank engine review id / cos / snippet
1 FAISS id=158 cos=0.688 Perfectly located just a short walk from the city ce
CHROMA id=158 cos=0.688 Perfectly located just a short walk from the city ce 2 FAISS
id=20 cos=0.678 Perfectly located just a short walk from the city ce CHROMA id=20
cos=0.678 Perfectly located just a short walk from the city ce 3 FAISS id=97
cos=0.670 Chose this place for its price. Perfectly located ju

```

```

CHROMA id=97 cos=0.670 Chose this place for its price. Perfectly located ju
4 FAISS id=24 cos=0.657 Perfectly located just a short walk from the city ce
CHROMA id=24 cos=0.657 Perfectly located just a short walk from the city ce
5 FAISS id=21 cos=0.632 Fantastic location right next to the metro and the m
CHROMA id=21 cos=0.632 Fantastic location right next to the metro and the m

```

```

=====
SPECIAL CASE - negation query: "didn't sleep well"
=====

```

Top-5 (FAISS) theme make-up:

```

poor sleep phrased via NEGATION (the target) : 5/5
night NOISE reviews (a common cause of poor sleep) : 0/5
TRUE opposite ('slept well', not also negated) : 0/5

```

Top-5 retrieved reviews:

1. id=240 [sleep_negation] I did not get a single good night's sleep here.
2. id=29 [sleep_negation;sleep_positive] I didn't sleep well at all during my stay. The blackout curtains
3. id=36 [sleep_negation] Sleep just wouldn't come and I was exhausted by morning. Would t
4. id=116 [sleep_negation] Honestly I barely slept the entire night. I'd happily book again
5. id=18 [sleep_negation;staff_rude] We stayed here for three nights. I didn't sleep well at all duri

Discussion: this is the hard case for semantic search - the concern is expressed by NEGATION and shares almost no words with the reviews. The all-MiniLM-L6-v2 model handles it correctly here: all 5/5 results genuinely describe poor or disturbed sleep, and 0/5 are true 'slept well' opposites - so the model matched MEANING, not surface words. (The single sleep_positive tag in the list belongs to id=29, a review that ALSO says 'I didn't sleep well', i.e. co-tagged, not a mismatch.) Negation is still a documented weakness of sentence embeddings in general, so at larger scale it is worth verifying. Crucially, FAISS and ChromaDB return the SAME ranking because they read identical vectors - negation behaviour is a property of the EMBEDDING model, not of the vector database.

Appendix B - Source Code:

benchmark_faiss_chroma.py # coding: utf-8 -*-

```

"""
Assessment Tool 3 - Q13 : Hotel Review Search - FAISS vs ChromaDB
=====

A travel site retrieves reviews matching a traveller's concern ("noise at night").
This script benchmarks two vector databases - FAISS and ChromaDB - on the SAME
sentence embeddings of 240 hotel reviews, over the SAME set of 12 semantic queries,
including the special negation case "didn't sleep well".

Pipeline:
reviews.csv -> embed once (all-MiniLM-L6-v2) -> index the SAME vectors in FAISS
(IndexFlatIP, cosine) and ChromaDB (HNSW, cosine) -> run 12 queries on each ->
benchmark indexing time, query latency, relevance@5, storage footprint -> compare
top-5 overlap on 3 queries -> discuss the negation special case.

Run: python benchmark_faiss_chroma.py
Outputs: prints a full report, and writes RESULTS.md + benchmark_output.txt
"""
import os
import csv
import time
import shutil
import warnings
from pathlib import Path

warnings.filterwarnings("ignore")
os.environ.setdefault("TOKENIZERS_PARALLELISM", "false")
os.environ.setdefault("ANONYMIZED_TELEMETRY", "False")

import numpy as np

HERE = Path(__file__).resolve().parent
CSV_PATH = HERE / "hotel_reviews.csv"
CHROMA_DIR = HERE / "chroma_db"
FAISS_FILE = HERE / "faiss_index.bin"
MODEL_NAME = "all-MiniLM-L6-v2"

```

```

TOP_K = 5
WARMUP = 3
REPEATS = 50 # latency is averaged over this many timed runs per query

# 12 semantic queries (>= 10 required). Each has a TARGET theme set used as ground
# truth for relevance@5 (a hit = the review's themes intersect the target).
QUERIES = [
    ("noise at night", {"noise_night"}),
    ("didn't sleep well", {"sleep_negation"}), # special case ("friendly and helpful
    staff", {"staff_friendly"}), ("clean and tidy room", {"clean"}),
    ("great location near the city center", {"location_good"}), ("poor breakfast
    quality", {"breakfast_bad"}), ("comfortable bed and a good night's sleep",
    {"bed_comfort", "sleep_positive"}), ("good value for money", {"value"}),
    ("slow wifi and bad internet", {"wifi_bad"}),
    ("rude reception staff", {"staff_rude"}),
    ("no hot water in the shower", {"shower_hot_water"}), ("beautiful
    view from the room", {"view"}),
]
COMPARE_QUERIES = [0, 1, 4] # indices compared top-5 side by side (incl. special case)

_report_lines = []

def out(line=""):
    print(line)
    _report_lines.append(line)

# ----- load data ---
def load_reviews():
    ids, texts, themes, hotels = [], [], [], []
    with CSV_PATH.open(encoding="utf-8") as f:
        for row in csv.DictReader(f):
            ids.append(int(row["id"]))
            hotels.append(row["hotel"])
            texts.append(row["review"])
            themes.append(set(row["themes"].split(";")))
    return ids, hotels, texts, themes

# ----- main -----
def main():
    ids, hotels, texts, themes = load_reviews()
    n = len(texts)

    # ---- embed once (same vectors feed BOTH databases) -----
    from sentence_transformers import SentenceTransformer
    out("Loading embedding model: %s" % MODEL_NAME)
    model = SentenceTransformer(MODEL_NAME)

    t0 = time.perf_counter()
    emb = model.encode(texts, normalize_embeddings=True, convert_to_numpy=True,
    batch_size=64).astype("float32")
    embed_time = time.perf_counter() - t0
    dim = emb.shape[1]

    q_texts = [q for q, _ in QUERIES]
    q_emb = model.encode(q_texts, normalize_embeddings=True,
    convert_to_numpy=True).astype("float32")

    out("=" * 74)
    out("SETUP")
    out("=" * 74)
    out("Reviews (records) : %d" % n)
    out("Embedding model : %s (dim=%d, cosine via normalised vectors)" % (MODEL_NAME, dim)) out("Embedding
    gen. time : %.3f s (shared by both DBs, not counted as index time)" % embed_time) out("Embeddings array
    size : %.2f MB (%d x %d float32)"
    % (emb.nbytes / 1e6, n, dim))
    out("Queries : %d | top-k = %d" % (len(QUERIES), TOP_K))
    out("")

    # ===== FAISS
    import faiss
    t0 = time.perf_counter()
    faiss_index = faiss.IndexFlatIP(dim) # inner product on normalised == cosine
    faiss_index.add(emb)
    faiss_index_time = time.perf_counter() - t0

    def faiss_search(qvec, k=TOP_K):
        D, I = faiss_index.search(qvec.reshape(1, -1), k)
        return list(I[0]), list(D[0]) # row positions, cosine similarities

    # ===== CHROMA
    import chromadb
    from chromadb.config import Settings
    if CHROMA_DIR.exists():
        shutil.rmtree(CHROMA_DIR)
    client = chromadb.PersistentClient(path=str(CHROMA_DIR),

```



```

settings=Settings(anonymized_telemetry=False)) t0 = time.perf_counter()
collection = client.create_collection(
name="hotel_reviews", metadata={"hnsw:space": "cosine"})
collection.add(
ids=[str(i) for i in ids],
embeddings=emb.tolist(),
documents=texts,
metadatas=[{"hotel": h, "themes": ";".join(sorted(t))}
for h, t in zip(hotels, themes)],
)
chroma_index_time = time.perf_counter() - t0

id_to_pos = {i: p for p, i in enumerate(ids)}

def chroma_search(qvec, k=TOP_K):
r = collection.query(query_embeddings=qvec.tolist(), n_results=k)
rids = [int(x) for x in r["ids"][0]]
sims = [1.0 - d for d in r["distances"][0]] # cosine distance -> similarity return
[id_to_pos[i] for i in rids], sims

# ----- latency (warm + timed)
def bench_latency(search_fn):
per_query_ms = []
for qv in q_emb:
for _ in range(WARMUP):
search_fn(qv)
t0 = time.perf_counter()
for _ in range(REPEATS):
search_fn(qv)
per_query_ms.append((time.perf_counter() - t0) / REPEATS * 1000.0) return
per_query_ms
faiss_ms = bench_latency(faiss_search)
chroma_ms = bench_latency(chroma_search)

# ----- relevance@5 (labels)
def precision_at_k(search_fn):
precs, results = [], []
for (qtext, target), qv in zip(QUERIES, q_emb):
pos, sims = search_fn(qv)
hits = [1 if (themes[p] & target) else 0 for p in pos]
precs.append(sum(hits) / len(hits))
results.append((qtext, pos, sims, hits))
return precs, results

faiss_prec, faiss_res = precision_at_k(faiss_search)
chroma_prec, chroma_res = precision_at_k(chroma_search)

# ----- storage/mem
faiss.write_index(faiss_index, str(FAISS_FILE))
faiss_disk = FAISS_FILE.stat().st_size
faiss_ram = faiss_index.ntotal * dim * 4 # flat index RAM (bytes) chroma_disk =
sum(p.stat().st_size for p in CHROMA_DIR.rglob("*") if p.is_file())

# ===== REPORT: TABLE 1
out("=" * 74)
out("TABLE 1 - BENCHMARK SUMMARY (FAISS vs ChromaDB, same embeddings)")
out("=" * 74)
out("%-32s | %14s | %14s" % ("Metric", "FAISS", "ChromaDB"))
out("-" * 66)
out("%-32s | %14s | %14s" % ("Index type", "IndexFlatIP", "HNSW (cosine)"))
out("%-32s | %14s | %14s" % ("Exact / Approximate", "Exact", "Approximate"))
out("%-32s | %13.2f | %13.2f" % ("Indexing time (ms)",
faiss_index_time * 1000, chroma_index_time * 1000))
out("%-32s | %13.3f | %13.3f" % ("Avg query latency (ms)",
np.mean(faiss_ms), np.mean(chroma_ms)))
out("%-32s | %13.3f | %13.3f" % ("Median query latency (ms)",
np.median(faiss_ms), np.median(chroma_ms)))
out("%-32s | %13.3f | %13.3f" % ("Mean relevance@5 (precision)",
np.mean(faiss_prec), np.mean(chroma_prec)))
out("%-32s | %12.1f | %12.1f" % ("On-disk storage (KB)",
faiss_disk / 1024, chroma_disk / 1024))
out("%-32s | %12.1f | %12s" % ("Index RAM estimate (KB)",
faiss_ram / 1024, "n/a (HNSW)"))
out("")

# ===== REPORT: TABLE 2
out("=" * 74)
out("TABLE 2 - PER-QUERY RELEVANCE@5 and LATENCY")
out("=" * 74)
out("%-42s | %5s %5s | %7s %7s" %
("Query", "F.P@5", "C.P@5", "F.ms", "C.ms"))
out("-" * 72)
for i, (qtext, _t) in enumerate(QUERIES):
tag = " " if i == 1 else ""
out("%-42s | %5.2f %5.2f | %7.3f %7.3f" %
((qtext + tag)[:42], faiss_prec[i], chroma_prec[i],
faiss_ms[i], chroma_ms[i]))
out("( * = special negation case) F=FAISS C=ChromaDB P@5=precision@5")

```

```

out("")

# ===== REPORT: TABLE 3
out("=" * 74)
out("TABLE 3 - TOP-5 OVERLAP between FAISS and ChromaDB (3 queries)")
out("=" * 74)
for qi in COMPARE_QUERIES:
    qtext = QUERIES[qi][0]
    fpos, fs = faiss_res[qi][1], faiss_res[qi][2]
    cpos, cs = chroma_res[qi][1], chroma_res[qi][2]
    fset, cset = set(ids[p] for p in fpos), set(ids[p] for p in cpos)
    inter = fset & cset
    jacc = len(inter) / len(fset | cset)
    out("")
    out("Query: \"%s\" \"%s\" % (qtext, " [SPECIAL NEGATION CASE]" if qi == 1 else ""))
    out("overlap = %d/5 reviews Jaccard = %.2f" % (len(inter), jacc))
    out("rank, \"engine\", \"review id / cos / snippet\")) for rank in range(TOP_K):
        fp, cp = fpos[rank], cpos[rank]
        out(" %2d FAISS id=%3d cos=%.3f %s" %
            (rank + 1, ids[fp], fs[rank], texts[fp][:52]))
        out(" CHROMA id=%3d cos=%.3f %s" %
            (ids[cp], cs[rank], texts[cp][:52]))
    out("")

# ===== SPECIAL CASE ANALYSIS
out("=" * 74)
out("SPECIAL CASE - negation query: \"didn't sleep well\"")
out("=" * 74)
qi = 1
pos = faiss_res[qi][1]
tallies = [themes[p] for p in pos]
n_neg = sum(1 for t in tallies if "sleep_negation" in t)
n_noise = sum(1 for t in tallies if "noise_night" in t)
# a TRUE opposite = "slept well" review that is NOT also a poor-sleep review
n_opp = sum(1 for t in tallies if "sleep_positive" in t and "sleep_negation" not in t)
out("Top-5 (FAISS) theme make-up:")
out("poor sleep phrased via NEGATION (the target) : %d/5" % n_neg)
out("night NOISE reviews (a common cause of poor sleep) : %d/5" % n_noise)
out("TRUE opposite ('slept well', not also negated) : %d/5" % n_opp)
out("Top-5 retrieved reviews:")
for r, p in enumerate(pos, 1):
    out(" %d. id=%d [%s] %s" % (r, ids[p], ";".join(sorted(themes[p])), texts[p][:64]))
out("")
out("Discussion: this is the hard case for semantic search - the concern is")
out("expressed by NEGATION and shares almost no words with the reviews. The")
out("all-MiniLM-L6-v2 model handles it correctly here: all %d/5 results genuinely" % n_neg)
out("describe poor or disturbed sleep, and %d/5 are true 'slept well' opposites" % n_opp)
out("so the model matched MEANING, not surface words. (The single sleep_positive")
out("tag in the list belongs to id=29, a review that ALSO says 'I didn't sleep'")
out("well', i.e. co-tagged, not a mismatch.) Negation is still a documented")
out("weakness of sentence embeddings in general, so at larger scale it is worth")
out("verifying. Crucially, FAISS and ChromaDB return the SAME ranking because")
out("they read identical vectors - negation behaviour is a property of the")
out("EMBEDDING model, not of the vector database.")
out("")

# ===== SAVE (HERE / "benchmark_output.txt")
write_results_md(n, dim, embed_time, emb, faiss_index_time, chroma_index_time,
    faiss_ms, chroma_ms, faiss_prec, chroma_prec, faiss_disk, chroma_disk,
    faiss_ram, ids, texts, themes, faiss_res, chroma_res)
out("Saved: benchmark_output.txt and RESULTS.md")

def md_table(headers, rows):
    s = "| " + " | ".join(headers) + " |\n"
    s += "| " + " | ".join("----" for _ in headers) + " |\n"
    for r in rows:
        s += "| " + " | ".join(str(x) for x in r) + " |\n"
    return s

def write_results_md(n, dim, embed_time, emb, fi_t, ci_t, fms, cms, fp, cp,
    fdisk, cdisk, fram, ids, texts, themes, fres, cres):
    import numpy as np
    lines = []
    lines.append("# Results - FAISS vs ChromaDB on 240 hotel reviews\n")
    lines.append("Embedding model: **all-MiniLM-L6-v2** (384-dim, cosine). The **same** "
        "embeddings are indexed in both databases. Corpus: 240 synthetically "
        "generated hotel reviews (seed=42), 12 semantic queries, k=5.\n")
    lines.append("## Table 1 - Benchmark summary\n")
    lines.append(md_table(
        ["Metric", "FAISS", "ChromaDB"],
        [
            ["Index type", "IndexFlatIP (exact)", "HNSW (approximate)"],
            ["Indexing time (ms)", "%.2f" % (fi_t*1000), "%.2f" % (ci_t*1000)],
            ["Avg query latency (ms)", "%.3f" % np.mean(fms), "%.3f" % np.mean(cms)],
            ["Median query latency (ms)", "%.3f" % np.median(fms), "%.3f" % np.median(cms)],
            ["Mean relevance@5", "%.3f" % np.mean(fres), "%.3f" % np.mean(cres)],
            ["On-disk storage (KB)", "%.1f" % (fdisk/1024), "%.1f" % (cdisk/1024)],
            ["Index RAM estimate (KB)", "%.1f" % (fram/1024), "n/a (HNSW graph)"],
            ["Embedding gen time (s, shared)", "%.3f" % embed_time, "%.3f" %

```

```

embed_time]])) lines.append("\n## Table 2 - Per-query relevance@5 and latency\n")
rows = []
for i, (q, _) in enumerate(QUERIES):
    rows.append([q + ("*" if i == 1 else ""), "%.2f" % fp[i], "%.2f" % cp[i],
"%f" % fms[i], "%.3f" % cms[i]])
lines.append(md_table(["Query", "FAISS P@5", "Chroma P@5", "FAISS ms", "Chroma ms"], rows))
lines.append("\n`*` = special negation case. P@5 = precision@5 using theme labels.\n")
lines.append("\n## Table 3 - Top-5 overlap (3 queries)\n")
for qi in COMPARE_QUERIES:
    q = QUERIES[qi][0]
    fset = set(ids[p] for p in fres[qi][1])
    cset = set(ids[p] for p in cres[qi][1])
    inter = fset & cset
    jacc = len(inter) / len(fset | cset)
    lines.append("**\n%s\n** - overlap %d/5, Jaccard %.2f\n" %
(q, " (special negation case)" if qi == 1 else "", len(inter), jacc)) rr = []
for rank in range(TOP_K):
    fp_ = fres[qi][1][rank]
    cp_ = cres[qi][1][rank]
    rr.append([rank+1,
"id %d - %s" % (ids[fp_], texts[fp_][:40])],
"id %d - %s" % (ids[cp_], texts[cp_][:40])])
lines.append(md_table(["rank", "FAISS top-5", "ChromaDB top-5"], rr))
lines.append("")
(HERE / "RESULTS.md").write_text("\n".join(lines), encoding="utf-8")

if __name__ == "__main__":
    main()

```

Appendix C - Source Code: build_dataset.py

```

# -*- coding: utf-8 -*-
"""
Assessment Tool 3 - Q13 : Hotel Review Search (FAISS vs ChromaDB)
Step 1 of the pipeline - build the dataset.

Creates hotel_reviews.csv with 240 hotel-review text records.

Dataset source: SYNTHETICALLY GENERATED by this script (fixed seed = 42) so the
submission is fully self-contained and reproducible, and so every review carries a
known 'themes' label. Those labels are used later as ground truth to compute top-k
relevance (precision@5) objectively, instead of fragile keyword matching.

Each review is stitched from themed sentence pools that use varied wording (e.g.
"noisy street", "loud music", "thin walls") so that *semantic* search - not exact
keyword match - is what makes retrieval work. This is what lets a query like
"noise at night" or the negation query "didn't sleep well" retrieve the right
reviews even when they share no words with the review text.
"""
import csv
import random
from pathlib import Path

random.seed(42)
HERE = Path(__file__).resolve().parent
N_REVIEWS = 240

# ----- theme pools -----
# Each theme -> several differently-worded sentences (semantic variety, low lexical
# overlap) so the sentence-embedding model has to do the work.
THEMES = {
    "noise_night": [
        "The traffic noise at night kept me awake for hours.",
        "Loud music from the bar downstairs went on well past midnight.",
        "Thin walls meant I could hear every conversation next door after dark.",
        "Street noise poured in through the windows the whole night.",
        "Honking cars and sirens outside made the evenings really loud.",
    ],
    "sleep_negation": [ # poor sleep expressed via NEGATION (the special case)
        "I didn't sleep well at all during my stay.",
        "I couldn't sleep a wink because of the disturbances.",
        "Honestly I barely slept the entire night.",
        "I did not get a single good night's sleep here.",
        "Sleep just wouldn't come and I was exhausted by morning.",
    ],
    "sleep_positive": [
        "I slept like a baby every single night.",
        "The blackout curtains gave me the best sleep I've had in ages.",
        "It was so peaceful that I fell asleep the moment I lay down.",
    ],
    "clean": [
        "The room was spotless and had clearly been freshly cleaned.",
        "Immaculate bathroom, crisp white linens, not a speck of dust.",
        "Everything was hygienic and sparkling from floor to ceiling.",
    ],
    "dirty": [
        "The carpet was stained and the bathroom smelled musty.",
        "I found hair in the sink and a layer of dust on the shelves.",
        "The sheets looked grubby and the towels were far from fresh.",
    ],
    "staff_friendly": [
        "The reception staff were warm, welcoming and incredibly helpful.",
        "The front desk team went out of their way to make us feel at home.",
        "Every member of staff greeted us with a genuine smile.",
    ],
    "staff_rude": [
        "The receptionist was rude and dismissive when we asked for help.",
        "Staff seemed annoyed every time we made a simple request.",
        "The front desk was cold and unhelpful throughout our stay.",
    ],
    "location_good": [

```

```

        "Perfectly located just a short walk from the city centre.", - "Fantastic
location right next to the metro and the main square.", "You can reach
all the major sights on foot from the front door.", ],
    "location_bad": [
        "The hotel was far from everything and hard to reach.", "It's a long,
awkward journey from here to anything worth seeing.", "The area felt
isolated and miles away from the attractions.", ],
    "breakfast_good": [
        "The breakfast buffet was delicious with an enormous choice.",
        "Mornings started with fresh pastries, fruit and great coffee.", ],
    "breakfast_bad": [
        "Breakfast was cold, greasy and thoroughly disappointing.", "The morning
spread was stale and the coffee tasted watered down.", ],

```

```

"bed_comfort": [
    "The bed was wonderfully comfortable and generously sized.", "Sinking into
that soft, cosy mattress was the highlight of the day.", ],
"bed_bad": [
    "The mattress was lumpy and left my back aching.",
    "The bed was hard as a board and the pillows were flat.", ],
"value": [
    "Excellent value for money for everything you get.",
    "A real bargain - such a comfortable stay for so little.", ],
"overpriced": [
    "Overpriced for the tiny, tired room you actually get.", "Far too
expensive for such a basic, no-frills experience.", ],
"wifi_bad": [
    "The wifi was painfully slow and kept dropping out.",
    "The internet barely worked anywhere in the room.",
],
"wifi_good": [
    "Fast, reliable wifi reached every corner of the hotel.", ],
"shower_hot_water": [
    "There was no hot water in the shower for two mornings running.", "The
shower ran cold and the water pressure was pathetic.", ],
"view": [
    "We woke up to a stunning sea view from the balcony.",
    "The room looked out over a beautiful, quiet garden.",
],
"ac_temp": [
    "The air conditioning was noisy and never cooled the room.", ],
"parking": [
    "Convenient on-site parking made the whole trip effortless.", ],
"checkin": [
    "Check-in was quick, smooth and friendly.",
    "We queued for nearly an hour just to check in.",
],
}

# Weight primary-theme sampling so the key query targets are well represented,
# while every theme still appears several times.
PRIMARY_WEIGHTS = {
    "noise_night": 6, "sleep_negation": 6, "staff_friendly": 5, "staff_rude": 4,
    "clean": 5, "dirty": 4, "location_good": 5, "location_bad": 3,
    "breakfast_good": 4, "breakfast_bad": 4, "bed_comfort": 4, "bed_bad": 3,
    "value": 4, "overpriced": 3, "wifi_bad": 4, "wifi_good": 2, "shower_hot_water":
4, "view": 4, "sleep_positive": 3, "ac_temp": 3, "parking": 2, "checkin": 3,
}
PRIMARY_POOL = [t for t, w in PRIMARY_WEIGHTS.items() for _ in range(w)]

OPENERS = [
    "We stayed here for three nights.", "Booked this hotel for a weekend break.", "My
partner and I visited last month.", "Stopped here during a business trip.", "This
was our second time at this property.", "Chose this place for its price.", "Arrived
late after a long flight.", "Spent a week here on holiday.", "", "", "",
]
CLOSERS = [
    "Overall a memorable stay.", "Would think twice before returning.", "I'd
happily book again.", "Mixed feelings about the whole experience.", "Not sure
I'd recommend it to friends.", "A decent choice for the price.", "", "", ""
]
HOTELS = ["The Grand Plaza", "Seaside Inn", "City Central Hotel", "Riverside Lodge",
"Park View Suites", "The Old Harbour Hotel", "Sunrise Boutique", "Metro Comfort
Hotel", "Maple Court", "Azure Bay Resort"]

```

```

def build_review(rng):
    primary = rng.choice(PRIMARY_POOL)
    themes = [primary]
    parts = []
    opener = rng.choice(OPENERS)
    if opener:
        parts.append(opener)
    parts.append(rng.choice(THEMES[primary]))
    # 45% chance of a second, different theme to create multi-topic reviews
    if rng.random() < 0.45:
        secondary = rng.choice([t for t in THEMES if t != primary])
        parts.append(rng.choice(THEMES[secondary]))
        themes.append(secondary)
    closer = rng.choice(CLOSERS)
    if closer:
        parts.append(closer)
    text = " ".join(parts)
    return text, ";".join(themes), HOTELS[rng.randrange(len(HOTELS))]

def main():
    rng = random.Random(42)
    rows, seen = [], set()
    rid = 1
    # keep generating until we have N unique reviews
    while len(rows) < N_REVIEWS:
        text, themes, hotel = build_review(rng)
        if text in seen:
            continue
        seen.add(text)
        rows.append({"id": rid, "hotel": hotel, "review": text, "themes": themes})
        rid += 1

    out = HERE / "hotel_reviews.csv"
    with out.open("w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=["id", "hotel", "review", "themes"])
        w.writeheader()
        w.writerows(rows)

    # quick theme distribution report
    from collections import Counter
    dist = Counter(t for r in rows for t in r["themes"].split(";"))
    print("Wrote %d reviews -> %s" % (len(rows), out))
    print("Theme distribution (label = ground truth for relevance):")
    for t, c in sorted(dist.items(), key=lambda x: -x[1]):
        print(" %-18s %d" % (t, c))

if __name__ == "__main__":
    main()

```